sohamalloc

Progress report and briefing notes

What was done, what each step was based on, what helped, what did not and how it was changed, where other allocators are still better and why, and what comes next.

Prepared for Soham Basanwar, for the project review with his professor
Date 4 November 2026
Repository state branch sohamalloc-dev at 0b010d7; allocator code as of d2add24 (2026-10-23)
Companion sohamalloc-how-it-works.pdf, the slide deck for the meeting

Contents

1	Summary, and what to tell your professor
2	Timeline
3	Change by change: what, based on what, effect, what did not work
4	Defects found and fixed
5	Results: signal storms, benchmarks, memory return
6	Where other allocators are better, and why
7	What did not work: the negative results
8	Future steps
9	Preparing for the meeting: likely questions
10	How the work was done
A–C	References, key commits, where things are in the repository

1 Summary

sohamalloc is a general-purpose memory allocator written from scratch in C11: a drop-in replacement for malloc, free and C++ new/delete. Its thesis is that a thread-caching allocator can make **the whole malloc API async-signal-safe**, with no system call on the fast path, while staying as fast as the best allocators. It is also hardened by default (encoded free lists, invalid- and double-free detection) and returns freed memory to the kernel promptly.

Where it stands

Measure	Result	Note
Signal storms, final code	0 failures in 630 signal trials; 95% CI 0–0.006	The eight other allocators failed in 50–100% of 540 trials each
Benchmarks (24 workloads)	Combined score 1.142, tied first with mimalloc	Time 1.124 (mimalloc 1.063), RSS 1.162 (mimalloc 1.227); code of 2026-10-19, 3 repetitions
Memory return	≈ 50 MB resident after freeing 1 GiB	glibc 1,017, mimalloc 1,035, jemalloc 1,070, tcmalloc 628 MB
Cost of default hardening	1–7% on allocation microbenchmarks	glibc-simple 7%, cfrac 4%, alloc-test and espresso 1%
Defects	4 review rounds and one campaign: every confirmed critical and major defect fixed	Each fix has a regression test that fails before and passes after
Platforms	Linux x86-64, arm64, armhf, musl; Windows (soham_* API)	Full test matrix green on d2add24, including TSan, ASan and UBSan

The four gates before a paper

Gate	Status
1. Every known bug fixed, each with a fail-before / pass-after regression test	✓ **Met** on d2add24
2. A review round that confirms no critical or major finding	**Open.** Round 5 was started twice and stopped both times by usage limits before any reviewer finished
3. A signal-storm campaign with ≥ 30 trials per cell, confidence intervals, controls and documented limits	✓ **Met**: 0 / 630 on the final code; 0 / 1,800 control trials failed across all allocators
4. A tuned benchmark campaign with ≥ 10 repetitions and confidence intervals	**Open.** Not started; it needs a quiet machine set up with tools/bench-host.sh

Five things to tell your professor

- The problem is real: signal handlers reach malloc through syslog, logging and backtraces, and it has caused deadlocks and a root exploit (OpenSSH regreSSHion, CVE-2024-6387) [1].
- Every allocator we tested fails our signal storm in 50–100% of trials; sohamalloc failed none of 630.
- It does this without slowing down the common case: tied first on a 24-workload benchmark, with prompt memory return and hardening on by default.
- The mechanisms are mostly known (mimalloc, Temeraire, snmalloc, Safe-Linking); the new part is the documented signal-safety guarantee for the whole API in a fast thread-caching allocator, and the combination. It fits an integration or experience paper.
- Two gates are open: a clean review round and a tuned benchmark campaign with confidence intervals. The cost of the busy byte has not been measured cleanly yet.

2 Timeline

All dates are 2026. The project went from first survey to a tested allocator in about two days of active work; the following two weeks went into adversarial reviews, the signal-storm tool and campaigns, and fixes.

When	Phase	Outcome	Commits
Oct 18	Survey and first design (v1); five deep-research reports; self-critique; three competing designs	101 faults found in v1 (12 critical); designs A, B, C	ba06bb7, e7aaca2
Oct 19, 00–01 UTC	Two judges; synthesis into plan v2; red team; final plan v3; build spec	60 faults in plan v2 (9 critical), fixed in v3	61279d9, d295a53
Oct 19, 05–07 UTC	Core allocator; debugging round 1; performance round 1; memory return; tests and benchmark harness; campaign B1	Working allocator; B1 score 1.241 (2nd of 5)	67a8520 … 5ff3c75
Oct 19, 12–14 UTC	Debugging round 2; cross-platform; performance round 2; campaigns B2–B4; 32-bit fix	B4 score 1.142, tied first	7525796 … ead7c98
Oct 19–20	First review round's fixes; bitmap large-run placement	Double/interior free detection; 4 MiB malloc+free 10 → 0.19 µs	71f852c, ee1f70d
Oct 21	Review round 2 and fixes; sigstorm tool; review round 3 and fixes; signal-handler survey; storm campaign 1	5 + 6 defects; 6,480 storm trials	911d2ed … 7689931
Oct 22	C1-1 fix (handler mappings); verification-matrix script; tooling audit	Large handler allocations no longer fail	ae95875, 6e4527e
Oct 22–23	Review round 4 and fixes; aligned handler kind for the other allocators	16 findings recorded, all fixed or documented	d2add24, bc13653
Oct 23 – Nov 4	Review round 5 attempted twice (stopped by usage limits); storm rerun on the final code	0 / 630 signal trials failed on d2add24	0b010d7

3 Change by change

Each entry gives what changed, the published work it was based on (numbers refer to the references in appendix A), the measured effect, and what did not work and how it was changed. Commit hashes point into the repository's history.

3.1 Research and design process Oct 18–19 · ba06bb7, e7aaca2, 61279d9, d295a53

What changed	A staged process instead of coding straight away: survey; five deep-research reports; a self-critique of the first design; three competing designs; two judges; synthesis; a red team; final plan; build spec.
Based on	Allocator literature and source code: Hoard [2], jemalloc [3], mimalloc [4], TCMalloc Temeraire [5], snmalloc [6], the secure-allocator survey [7]. Stored in docs/research/01–05.
Effect	Design faults caught before any code existed: 101 in v1 (12 critical, 62 major) and 60 in plan v2 (9 critical). Examples: claimed novelties that had prior art (mm_cid indexing, lifetime prediction); a 47-bit address assumption (arm64 returns 48-bit addresses); a free-list encoding that could not be inverted; a check-then-use race in free(); a futex on a 1-byte field.
Did not work → changed	The first design's novelty claims did not survive the literature check. They were dropped, and the thesis narrowed to signal safety at competitive speed.
Evidence	docs/research/00-v1-critique.md; docs/SOHAMALLOC_PLAN.md §2.1 and §7.2; research log, timeline §6.1.

3.2 Core allocator v0.1 Oct 19 · 67a8520

What changed	4 GiB windows of 2 MiB hugeblocks and 64 KiB slices; out-of-line 192-byte slice records split by writer; a granule map for pointer lookup; 64 size classes; per-thread heaps; one-CAS remote frees; detach and claim of exited threads' pages; a static arena; encoded free-list links with exact block checks; a page heap with slice bitmaps and best-fit placement.
Based on	Out-of-line metadata: hardened_malloc [8], PartitionAlloc [9]. Address byte map: snmalloc [6]. Per-page remote lists: mimalloc [4], rpmalloc [10]. Hugepage-aware page heap: Temeraire [5]. Link encoding: Safe-Linking [11], PartitionAlloc, mimalloc [12]. Division-free multiple test: Lemire, Kaser and Kurz [13].
Effect	A working drop-in allocator in one session, passing a concurrency stress test with checksums under TSan, ASan, UBSan and on arm64, armhf and Windows.
Did not work → changed	The spec detached a page as soon as it filled, so other threads' frees could make it shared. That sent most frees through the remote CAS path: 0.160 s against mimalloc's 0.091 s on a mixed single-thread test. Replaced by owner-retained pages (3.3).
Evidence	docs/IMPLEMENTATION_NOTES.md §1–5 and §8.1; timeline §3.5.

3.3 Owner-retained pages and owner notification Oct 19 · 44e39e4; threshold in 273bcb9

What changed	Pages stay owned, on current, avail and full lists. A remote free to a full page notifies the owner through a lock-free stack (a WATCH flag in the remote-free word). The malloc fast path now covers every class up to 128 KiB.
Based on	mimalloc's free-list sharding and delayed free [4, 14]. The threshold follows Hoard's emptiness hysteresis [2] and mimalloc v3's 1/8 rule [15].
Effect	Instructions per call (callgrind, early build): free 63 → 54, malloc 44 → 41.
Did not work → changed	Notifying the owner at the first remote free made each refill yield a single block: 28% of mallocs took the slow path in alloc-test. Changed to a threshold of cap/8 for both remote and local frees: 0.07% slow path, alloc-test 2,228 → 1,966 ms, at the cost of at most cap/8 − 1 idle blocks per full page.
Evidence	IMPLEMENTATION_NOTES §5.2; timeline P0 and P2.

3.4 Fast-path engineering Oct 19 · c0d4822, 13bcf17

What changed	Owner and head offset loaded in parallel; the free word split into head and count; one busy byte gating both fast paths; inlined operator new; malloc and free entry points aligned to 64 bytes.
Based on	The measurement-bias literature [16] explained the code-placement effect; STABILIZER [17] shows how to evaluate around it.
Effect	glibc-simple went from 4.1 s to 2.6–2.9 s from the alignment alone, with otherwise identical code.
Did not work → changed	Aligning our own entry points does not remove layout bias from comparisons: the other allocators and the benchmarks keep their own layouts. The tuned campaign (gate 4) should use layout randomisation. An early ablation (3 noisy runs) put the busy byte, link encoding and bounds check together at about 40% on a tiny-object loop, with each one inside the noise; the busy byte's cost still needs a proper measurement.
Evidence	Timeline P3 and the ablation table in §3.5; IMPLEMENTATION_NOTES §9.

3.5 Memory return Oct 19 · e70903f, 273bcb9, ee1f70d

What changed	Excess release (free resident memory above a dirty limit goes back at once); a recent-peak swing in that limit; runs that prefer resident hugeblocks; bitmap placement of large runs; decay checks on refill; release system calls outside the lock, batched with process_madvise.
Based on	Temeraire's demand-based HugeCache and TCMalloc's skip-subrelease [5, 18]; Go's scavenger [19]; jemalloc's decay and process_madvise batching [20, 21]; the stall reported when tcmalloc releases under its lock [22].
Effect	RSS after freeing 1 GiB: 1,038 → 50 MB (glibc 1,017, jemalloc 1,070, mimalloc 1,035, tcmalloc 628 MB). A 4 MiB malloc + free among 16k fragmented hugeblocks: 10 µs → 0.19 µs; 60k sequential 4 MiB allocations: 6.6 s → 0.43 s.
Did not work → changed	Excess release alone made malloc-large 2.88× slower than the best (1.8 million minor faults per run against 225k for glibc): 5–25 MiB buffers were released and re-faulted on every cycle. The recent-peak swing keeps memory a program is about to reuse: 1.18× glibc with the lowest RSS (B2), 1.38× of the best (B4). The resident-first scan then broke 32-bit targets (every allocation ≥ 2 MiB failed on armhf and Windows, D14), fixed with 64-bit scores in ead7c98.

Evidence	Timeline §3.5 RSS table, P1, D4, D14; commit message of ee1f70d.

3.6 Small-program footprint and C++ support Oct 19 · 273bcb9

What changed	C++ operators written in C under their mangled names, so the library no longer needs libstdc++; lazy metadata; a 4 KiB provisioning step.
Based on	mimalloc's alloc-override.c [23]; libstdc++'s nothrow operator [24].
Effect	Resident memory of a trivial program: 3,836 → 1,760 kB.
Did not work → changed	A throwing new_handler inside nothrow new called std::terminate, because C frames cannot catch. The nothrow forms now delegate to the C++ runtime's own nothrow operator, found once with dlsym(RTLD_NEXT).
Evidence	Timeline D10 and D11; IMPLEMENTATION_NOTES §8.7.

3.7 Hardening and invalid-free detection Oct 19–23 · 67a8520, 71f852c, c1a9791, d2add24

What changed	Encoded links with exact block checks; deterministic double- and interior-free detection; foreign-pointer detection through the granule map; exact-length walks of remote chains; a check that a dead thread's page has no block on both of its lists; frees into spare pages abort; a handler never writes into a pointer that may be foreign.
Based on	Safe-Linking [11], PartitionAlloc [9], mimalloc's secure mode [12], snmalloc's free-list protection [25], hardened_malloc [8], isoalloc [26].
Effect	1–7% cost on allocation microbenchmarks. Aborts on non-adjacent double frees, on double frees into remote lists and into pages of exited threads; glibc catches only the first kind.
Did not work → changed	Review round 2 found a double free on an exited thread's page that release by count missed (R2-3): fixed with an exact-length chain walk. Round 4 found one split across the page's two lists (R4-2): fixed with an overlap check before release or claim. Round 3 found that one heap's page keys come from one invertible generator stream (R3-6): documented, with a keyed derivation planned.
Evidence	docs/research-log/03-defects-and-faults.md, rounds 2–4; IMPLEMENTATION_NOTES §3.

3.8 Signal safety, the thesis feature

This feature changed the most, and every step came from a failure found by a test, a review or a measurement. The mechanism as built: a thread-local busy byte marks a thread as inside the allocator; a handler that finds it set runs in trymode, where every lock is a trylock, and allocates from a shared heap, then a static arena, then a mapping of its own; its frees use the lock-free remote path or are deferred until the interrupted call returns.

Step	Problem found	Change	Result
v0.1	Handler mallocs used only a never-reused static arena and ran out (D8)	Shared heap under trylock (trymode), arena as fallback (e70903f)	26,110 handler calls, 0 NULL

Step	Problem found	Change	Result
D13	Init, thread attach and exit, stats and heap checks took locks without the busy byte: a handler could wait on its own thread	Busy byte on every such path (273bcb9)	New test: 5 of 6 runs hung before, 15 of 15 passed after
R2-1	A refill in trymode called a blocking trim of empty pages: deadlock	Invariant H3 (no blocking lock in trymode), checked in debug builds (c1a9791)	Stall in 3 of 3 runs before, pass after; storms had missed it
R3-2	Aligned allocations in handlers still used only the arena: ENOMEM after about 384 KiB	Aligned family uses the shared heap (fd735dd)	13,108–15,622 NULL of about 41k calls → 0
R3-3	The arena counter also moved on failed requests and wrapped on 32-bit	Counter moves only when a block fits (CAS loop)	Counter bounded; armhf crash gone
R3-4	Memory that handlers allocated was never returned	Other threads' refills trim the shared heap; malloc_trim flushes it	16,703 KiB → 383 KiB cached; 0 after a flush
C1-1	Large handler requests used up the arena, then failed (about 45% NULL in a targeted test)	Requests above 16 KiB get a mapping of their own under trylock (ae95875)	Storm NULLs for large blocks: 13,838 → 31 of about 20 million
R4-3	Frees deferred by handlers could wait forever; those during thread exit leaked	Runs and solos freed at once by trylock; drains at the end of slow paths and thread exit (d2add24)	Regression test fails before, passes after

Based on. The property was first published for a fully lock-free allocator by Michael [27], and later LRMalloc [28]. The mechanism (a reentrancy flag, trylock-only allocation, a fallback pool, deferred frees) appears in Linux's bpf_mem_alloc [29] and kmalloc_nolock [30]. Special-purpose answers: AO_malloc [31], folly's ReentrantAllocator [32], Abseil's LowLevelAlloc with signals blocked [33]. The rules being broken: POSIX [34] and CERT SIG30-C [35]; the class of attack goes back to Zalewski [36].

A negative result that shaped the method. Randomized storms did not find the R2-1 deadlock: 9 runs of about 385,000 signals each all survived on the broken code. It needs three conditions at once, which a targeted test forces. Since then, claims rest on targeted tests, the debug check of H3 and reviews, with storms as supporting evidence reported with their trial counts.

3.9 fork() and initialisation Oct 21–23 · fd735dd, d2add24

What changed	Busy byte saved per thread across fork (R3-1); fork handlers registered at load by an ELF constructor, and fork-prepare finishes init before taking every lock (R4-1); init owned by a process id so a child can finish an init that fork cut off; the forking thread treated as holding every lock so atfork handlers can allocate (R4-4); the thread key deleted at unload (R4-5).
Effect	Concurrent forks: crash in 3 of 3 runs before, 1,000 clean rounds after. Fork during init: 0 hangs in 1,000 children, 800 of them under load.
Did not work → changed	The first R4-1 fix skipped locking when init had not run. Another thread could then finish init and take a lock just before the fork: 1 child in 40 still hung. The final fix always finishes init and always locks.
Evidence	IMPLEMENTATION_NOTES §7; defect ledger rounds 3 and 4.

3.10 Portability Oct 19 · 3d2dd95, 7525796, eabd154, 273bcb9, ead7c98

What changed	Smaller windows and no address hint under TSan (D1); 64-bit count-leading-zeros builtins on Windows (D2); thread exit that survives emulated TLS being destroyed first on mingw (D9); bcrypt linked on Windows (D12); 64-bit scores in the large-run scan (D14).
Effect	aarch64, armhf, musl and Windows suites all pass.
Did not work → changed	A methodology error (M1): the first cross-target runs had tested the C library's allocator, not sohamalloc. Per-target libraries are now built and passed explicitly to the runner.
Evidence	Timeline §6.2.

3.11 Tools and method Oct 19–23 · bench/, 911d2ed, 85f3b78, f3809c0, tools/verify-matrix.sh

What changed	A benchmark harness (a mimalloc-bench subset plus python, gcc, sqlite and redis; library snapshots with sha256); sigstorm, a signal-storm tool with a campaign runner, controls, Wilson intervals and resume; one script for the full verification matrix; multi-agent adversarial reviews with refuters; a research log.
Based on	mimalloc-bench [37]; Wilson score intervals [38].
Effect	Every claim in this report traces to a test, a data file or a commit.
Did not work → changed	A regression test first failed to reproduce R3-1: GCC had removed the test's own malloc/free pairs (built without -fno-builtin-malloc), and a sleeping barrier let one fork finish before the other began. Rebuilt with the runner's flags and spin-started, it fails on the old code every time.

4 Defects found and fixed

Source	Found	Main examples	Status
Design reviews (before code)	101 faults in v1, 60 in plan v2 (21 critical)	Prior-art novelty claims, 47-bit address assumption, non-invertible encoding, free() race	Addressed in plan v3
Testing and measurement, Oct 19	D1–D14, P0–P4, M1, C1, O1	TSan window collision, Windows builtins, memory not returned, signal deadlock D13, 32-bit large runs	Fixed except P4 (larson gap) and O1
First review, Oct 19–20	several	Double and interior frees, signal-safe solo paths, notify and pending races	Fixed (71f852c)
Review round 2, Oct 21	2 critical, 1 major, 2 minor	Handler deadlock R2-1, exited-thread double free R2-3, notify stranding after fork R2-2	Fixed (c1a9791)
Review round 3, Oct 21	1 critical, 3 major, 2 minor	Concurrent fork crash R3-1, handler aligned ENOMEM R3-2, arena wrap R3-3, shared heap never released R3-4	Fixed (fd735dd); minors documented
Storm campaign 1, Oct 21	1 major	Large handler allocations returned NULL (C1-1)	Fixed (ae95875)
Review round 4, Oct 22–23	2 critical, 4 major, 10 minor	Fork during init, double free split across lists, deferred frees, atfork allocation, unload crash	Fixed or documented (d2add24)
Review round 5	—	Started twice, stopped by usage limits	Not completed

Every confirmed critical and major defect was reproduced before it was fixed, and each fix has a regression test that fails on the old code and passes on the new. The before/after records are in docs/research-log/data/regress-before-after/. Minor findings that were not fixed are listed as limitations in docs/IMPLEMENTATION_NOTES.md §9.

5 Results

5.1 Signal storms

Each trial runs 1, 2 or 4 workload threads for 5 s while a signaller sends SIGUSR1 to each in turn every 0–40 µs, and the handler calls the allocator. Outcomes: survived; stall (a thread made no progress for 2 s); corruption (contents changed or live blocks overlapped); crash. 30 trials per cell; 95% Wilson intervals. Machine: Intel Core Ultra 9 185H, 22 CPUs, Ubuntu 26.04.

Allocator	Trials	P(fail) [95% CI]	Stall	Crash	Corrupt	aligned kind
sohamalloc (final code)	**630**	**0.000 [0.000, 0.006]**	**0**	**0**	**0**	**0 / 90**
tbbmalloc	540	0.500 [0.458, 0.542]	28	242	0	90 / 90
scudo	540	0.633 [0.592, 0.673]	293	5	44	39 / 90
mimalloc	540	0.661 [0.620, 0.700]	0	300	57	85 / 90

Allocator	Trials	P(fail) [95% CI]	Stall	Crash	Corrupt	aligned kind
snmalloc	540	0.830 [0.796, 0.859]	77	267	103	76 / 90
jemalloc	540	0.867 [0.835, 0.893]	373	85	10	90 / 90
glibc	540	0.896 [0.868, 0.919]	484	0	0	90 / 90
tcmalloc	540	1.000 [0.993, 1.000]	373	151	16	90 / 90
hardened_malloc	540	1.000 [0.993, 1.000]	540	0	0	90 / 90

Others: campaign 1 (2026-10-21), six allocating handler kinds × 3 thread counts × 30 trials; aligned kind run separately (2026-10-22/23). sohamalloc: rerun on d2add24 with all seven allocating kinds. Controls (no signals, or a handler that does not allocate): 1,800 trials, none failed.

Reading the failure modes. A stall means a thread stopped making progress, consistent with a handler waiting for a lock that its own interrupted thread holds; glibc and hardened_malloc fail only this way. Crashes and corruption are consistent with a handler changing a thread cache or free list in the middle of an update; mimalloc, tbbmalloc and snmalloc fail mostly this way. Neither is a bug in those allocators: POSIX does not ask them to be signal-safe.

sohamalloc across versions

Code	Signal trials	Failed	NULL results, large-block handler	Churn	Aligned
c1a9791 (campaign 1)	540	0	13,838 of 20,305,876	12	(kind not yet in the tool)
fd735dd (partial rerun)	456	0	6,743	70	39
d2add24 (final)	**630**	**0**	**31 of 19,941,425**	**2**	**0**

A NULL from a handler is the documented limit (the locks it needs were held and both fallbacks failed), not a crash. The drop from 13,838 to 31 is the C1-1 fix.

5.2 Benchmarks

Allocator	Time geomean	RSS geomean	Score
sohamalloc	**1.124**	**1.162**	**1.142**
mimalloc 2.1.2	**1.063**	1.227	**1.142**
glibc 2.39	1.491	1.092	1.276
jemalloc 5.3.0	1.333	1.355	1.344
tcmalloc (gperftools 2.15)	1.370	1.537	1.451

Campaign B4: 24 workloads, 5 allocators, 3 repetitions, on a 4-vCPU cloud VM; each workload's time and peak RSS normalised to the best allocator; score = geometric mean of the two geomeans; lower is better. sohamalloc was fastest on cfrac, cache-scratch1, rptestN, python-malloc and gxx, and never the most memory-hungry; slowest on barnes, cache-thrashN and python.

Progress from B1 to B4 (factor against the best allocator)

Workload	B1	B4	Workload	B1	B4
malloc-large	2.88	1.38	cache-scratchN	1.00	1.32 (worse)
sh8benchN	1.71	1.07	cache-thrashN	1.10	1.31 (worse)
glibc-thread	1.54	1.05	glibc-simple	1.17	1.25 (worse)
larsonN-sized	1.40	1.20	barnes	1.00	1.20 (worse)
alloc-test1	1.40	1.21	larsonN	1.24	1.07
xmalloc-testN	1.38	1.23	Score	1.241	1.142

Absolute times moved for every allocator between B1 and B4 (B1 ran while test agents loaded the same 4 CPUs), and run-to-run spread reached 117–171% on cache-scratchN, so these factors indicate direction, not precise size. The tuned campaign will settle them.

5.3 Memory return

Allocator (RSS, MB)	Peak	After free	After 2 s	After malloc_trim
glibc	1,017	1,017	1,017	9
jemalloc	1,070	1,070	1,070	1,070
mimalloc	1,035	1,035	1,035	1,035
tcmalloc	1,047	628	628	628
sohamalloc before excess release	1,038	1,038	1,038	14
sohamalloc with excess release	**1,038**	**50**	**50**	**14**

A program allocates about 1 GiB, frees it, waits 2 s, then calls malloc_trim (2026-10-19).

6 Where other allocators are better, and why

Being clear about these makes the strengths more believable. Each item gives the evidence, the reason as far as it is understood, and what could be done.

Raw speed: mimalloc

Time geomean 1.063 against 1.124. Our fast paths take about 45 instructions for malloc and 79 for free (callgrind, glibc-simple); mimalloc's about 18 and 29. The extra work is the security and safety checks: decoding and checking each link, clearing it, the foreign-pointer test, the owner check and the busy byte. Options: measure each part with confidence intervals, then trim instructions without dropping the guarantees.

Producer/consumer (larson): mimalloc

83.0 million against 68.6 million operations per second, about 1.2×. Every remote free is a CAS plus notification bookkeeping, and collecting a remote chain validates every node. mimalloc's delayed free is lighter. Open item P4.

Random-order frees: mimalloc

alloc-test1 1.21×, xmalloc-testN 1.23×. The cap/8 threshold means more refills, and each refill validates the chains it collects.

barnes, cache-thrashN, cache-scratchN, python

1.20–1.32×, and these got worse from B1 to B4. The cache tests run for about 0.5 s, close to timer resolution, with spread up to 117%; python was slowest of the five in B4. Not explained yet: the tuned campaign has to check whether this is noise, code placement or a real cost.

Pure fast-path microbenchmarks: glibc, mimalloc

glibc-simple 1.25×: a loop of malloc and free that shows the longer fast path directly, and reacts strongly to code placement.

Large reused buffers: tcmalloc

malloc-large 1.38× of the best: prompt release still returns some memory that is about to be reused. This is the price of the lowest RSS.

Memory of small processes: glibc

glibc has the lowest RSS geomean (1.092 against 1.162). Windows, slice records, the granule map and per-thread heaps cost a fixed amount; the footprint work cut a trivial program from 3,836 to 1,760 kB, but small C programs still use more than with glibc.

Tight address-space limits: glibc

Under ulimit -v 128 MiB about 60 MiB is usable against about 100 MiB with glibc, because reserved windows and the granule map count against the limit (R3-5).

Idle threads: jemalloc, TCMalloc

They can release memory from a background thread [20]. sohamalloc has none: an idle thread keeps its pages until it exits or allocates again. The planned membarrier-based lease is not implemented.

Hardening depth: snmalloc, hardened_malloc

snmalloc's signed back-pointers catch more corruption [25]; hardened_malloc and isoalloc isolate more. Our per-page keys come from one generator stream per heap (R3-6), and the secure tier's cost and detection rate are unmeasured.

Progress guarantee: Michael (2004), LRMalloc

Fully lock-free allocators [27, 28] never fail for lack of a lock. Our handler path is lock-free too but can return NULL when the locks it needs are held.

Placement policy at scale: TCMalloc Temeraire

Temeraire ranks hugepages by longest free range and allocation count, and argues against plain best-fit [5], which is what sohamalloc uses.

7 What did not work: the negative results

These are worth saying out loud; each one changed the design or the method.

- **Detach-on-full** (from the spec) sent most frees down the remote path, 0.160 s against mimalloc's 0.091 s: replaced by owner-retained pages.
- **Notifying the owner at the first free** put 28% of mallocs on the slow path: replaced by the cap/8 threshold.

- **A static arena as the only handler memory** ran out (D8), then failed for aligned (R3-2) and large (C1-1) requests: replaced step by step by the shared heap, aligned support and handler mappings.
- **Excess release on its own** made malloc-large 2.88× slower: fixed with the recent-peak swing.
- **The first fix for fork during init** still hung 1 child in 40: replaced by always finishing init and locking.
- **Randomized storms missed a real deadlock** (R2-1): targeted tests and the H3 debug check were added; storm results are reported with their trial counts.
- **A regression test that did not reproduce its defect** (compiler removed the calls; a sleeping barrier hid the race): fixed so it fails on the old code every time.
- **Cross-target runs that tested the wrong allocator** (M1): per-target builds passed explicitly.
- **An optimisation that broke 32-bit targets** (D14): found by the cross-target suites, fixed with 64-bit arithmetic.
- **Aligning entry points** fixed our own code-placement sensitivity but not measurement bias in comparisons: the tuned campaign needs layout randomisation.
- **Review workflows cut off by usage limits**: round 3's verifiers, a third of round 4's agents, and round 5 twice. This is why gate 2 is still open.

8 Future steps

#	Step	Why	What it needs
1	Review round 5, then repeat until clean	Gate 2	A session with enough usage; the script and frozen libraries are ready
2	Measure the busy byte's cost	The thesis claims safety at competitive speed; the cost must be shown	Ablation builds, ≥ 10 repetitions, confidence intervals
3	Tuned benchmark campaign on the final code	Gate 4; settles the open benchmark questions	A quiet machine (tools/bench-host.sh tune), layout randomisation
4	Close the larson gap (P4)	Cross-thread frees are the weakest workload	Profiling of remote free and chain collection
5	Return idle threads' memory	No background thread today	The planned membarrier lease, or an optional purge thread
6	Keyed per-page keys (R3-6); smaller footprint under RLIMIT_AS (R3-5)	The two documented minor findings	Design and tests
7	Model-check the page protocols	Storms are evidence, not proof	TLA+ or GenMC models of remote free, notify and detach/claim
8	Real-world signal workloads; measure the secure tier	Reviewers will ask for both	Crash reporters, profilers, logging libraries; S1 cost and detection rate
9	Finish the research log, then the paper	After the four gates	An integration or experience paper, for example at ISMM

9 Preparing for the meeting: likely questions

“Is any of this new?”

Most mechanisms are known, and the deck says so with citations. What is new is a documented async-signal-safety guarantee for the whole malloc API in a user-space, thread-caching, general-purpose allocator, at competitive speed, together with default hardening and prompt memory return. That is an integration or experience contribution, not a new algorithm.

“How do you know it is signal-safe?”

Three invariants (H1–H3), with H3 checked in debug builds; a regression test for every defect found; four adversarial review rounds; and a storm campaign with confidence intervals (0 of 630 against 50–100% for the others). It is evidence, not proof: storms once missed a deadlock, which is why model checking is planned.

“What does the safety cost?”

Not measured cleanly yet. An early, noisy ablation put the busy byte, link encoding and bounds check together at about 40% on a tiny-object loop, yet the whole allocator ties mimalloc on the benchmark score. A proper measurement is the next step.

“Why is mimalloc still faster?”

Its fast paths are about 2.5 times shorter because they skip our checks; larson is 1.2× because our cross-thread frees do more bookkeeping.

“Can malloc in a handler fail?”

Yes, by design: it returns NULL rather than wait for a lock. On the final code that happened 31 times in 19.9 million large-block handler calls, and never for small blocks.

“How did you use AI?”

Section 10 has the facts. The goals, thesis, priorities and gates were mine; AI-assisted tools did most of the hands-on research, coding, testing and reviewing; every claim is checked against tests and raw data, and the research log records each step.

“What is missing before a paper?”

A clean review round (gate 2), the tuned benchmark campaign with confidence intervals (gate 4), and the busy-byte cost measurement.

10 How the work was done

The research log (docs/research-log/01-timeline.md §0.2) records the method. In short:

- **The author directed the work in natural language:** the goals, the thesis, the priorities and the four acceptance gates. Key choices were offered as explicit decisions and recorded.
- **AI-assisted tooling did the hands-on work:** an interactive AI coding assistant, plus orchestrated multi-agent workflows in which agents had assigned roles (researcher, critic, designer, judge, red-teamer, implementer, test engineer, benchmark engineer, reviewer, refuter). This covered literature research, critique, design, implementation, testing, benchmarking and documentation.
- **Evidence before claims:** a review finding counts only once a reproducer confirms it; every fix needs a regression test that fails before and passes after; allocator code is committed only after the full matrix passes; raw benchmark and storm data are kept in the repository with library checksums.

- **Interruptions:** several workflows were stopped by usage limits, so some review stages never completed. The ledger marks which findings were verified and which were not.

Saying this plainly in the meeting is the safe course: the contribution is the direction, the evaluation and the evidence, and the repository shows exactly how each result was produced.

A References

[1] Qualys Threat Research Unit. regreSSHion: remote unauthenticated code execution in OpenSSH server (CVE-2024-6387). 2024.

[2] E. D. Berger, K. S. McKinley, R. D. Blumofe, P. R. Wilson. Hoard: A Scalable Memory Allocator for Multithreaded Applications. ASPLOS 2000.

[3] J. Evans. A Scalable Concurrent malloc(3) Implementation for FreeBSD. BSDCan 2006.

[4] D. Leijen, B. Zorn, L. de Moura. Mimalloc: Free List Sharding in Action. APLAS 2019 (MSR-TR-2019-18).

[5] A. H. Hunter, C. Kennelly, P. Turner, D. Gove, T. Moseley, P. Ranganathan. Beyond malloc efficiency to fleet efficiency: a hugepage-aware memory allocator. OSDI 2021.

[6] P. Lietar, T. Butler, S. Clebsch, S. Drossopoulou, J. Franco, M. J. Parkinson, A. Shamis, C. M. Wintersteiger, D. Chisnall. snmalloc: A Message Passing Allocator. ISMM 2019.

[7] B. Novkovic et al. SoK: Secure Memory Allocation. 2021.

[8] GrapheneOS hardened_malloc README.

[9] Chromium PartitionAlloc: design document and encoded_next_freelist.h.

[10] M. Jansson. rpmalloc README.

[11] Check Point Research (E. Itkin). Safe-Linking: Eliminating a 20 year-old malloc() exploit primitive. 2020.

[12] mimalloc source, include/mimalloc/internal.h: per-page keys for encoded free-list pointers.

[13] D. Lemire, O. Kaser, N. Kurz. Faster remainder by direct computation: applications to compilers and software libraries. Software: Practice and Experience 49(6), 2019.

[14] mimalloc source, src/free.c (v2): delayed free for pages in the full queue.

[15] mimalloc source, src/alloc/free.c (v3): owned bit; re-abandon once a page is below 7/8 used.

[16] T. Mytkowicz, A. Diwan, M. Hauswirth, P. F. Sweeney. Producing wrong data without doing anything obviously wrong! ASPLOS 2009.

[17] C. Curtsinger, E. D. Berger. STABILIZER: Statistically Sound Performance Evaluation. ASPLOS 2013.

[18] TCMalloc source, tcmalloc/huge_page_filler.h (skip-subrelease; release from the emptiest hugepage).

[19] Go runtime, mgcscavenge.go (scavenger goals, retainExtraPercent).

[20] jemalloc manual: opt.dirty_decay_ms, opt.background_thread, opt.process_madvise_max_batch.

[21] jemalloc pull request #2794, “Implement process_madvise support”, merged 2025-03-07.

[22] MongoDB SERVER-31417: tcmalloc holds an internal lock while releasing memory with madvise.

[23] mimalloc source, src/alloc-override.c: C++ operators defined in C under mangled names.

[24] libstdc++ libsupc++/new_opnt.cc: nothrow operator new as try/catch around ::operator new (LWG 206).

[25] snmalloc documentation: Protecting free lists from user corruption (FreelistProtection.md).

[26] isoalloc README (struct/isoalloc).

[27] M. M. Michael. Scalable Lock-Free Dynamic Memory Allocation. PLDI 2004, pp. 35–46.

[28] R. Leite, R. Rocha. LRMalloc: A Modern and Competitive Lock-Free Dynamic Memory Allocator. 2018.

[29] Linux kernel/bpf/memalloc.c: BPF any-context allocator (per-CPU active counter, deferred frees).

[30] A. Starovoitov. slab: Re-entrant kmalloc_nolock(), v5 patch series, September 2025 (LWN).

[31] libatomic_ops README_malloc (AO_malloc: allocation from signal handlers).

[32] folly memory/ReentrantAllocator.h.
[33] Abseil base/internal/low_level_alloc.cc (async-signal-safe arenas: signals blocked around a spinlock).
[34] POSIX.1-2024 and the Linux signal-safety(7) manual page: the async-signal-safe function list.
[35] SEI CERT C Coding Standard, rule SIG30-C: call only asynchronous-safe functions within signal handlers.
[36] M. Zalewski. Delivering signals for fun and profit. 2001.
[37] mimalloc-bench benchmark suite, pinned at commit 3913826 (2026-10-17).
[38] E. B. Wilson. Probable inference, the law of succession, and statistical inference. JASA 22(158), 1927.

Further references (50 in all, with links) are in docs/research-log/05-novelty-analysis.json, and the real-world signal-handler cases in docs/research/06-signal-handler-allocation-survey.md.

B Key commits

Commit	Date	What it did
67a8520	Oct 19	Core allocator v0.1
44e39e4	Oct 19	Owner-retained pages with remote-free notification; leaner fast paths
e70903f	Oct 19	Excess release; handler mallocs use the shared heap; check_heap fix
273bcb9	Oct 19	C++ operators in C; refill and release hysteresis; busy discipline for init/attach
13bcf17	Oct 19	One busy byte for both fast paths; inline new; aligned hot entries
ead7c98	Oct 19	Large-run placement fixed on 32-bit and LLP64
71f852c	Oct 19	First review's fixes: double/interior frees, signal-safe solo paths, races
ee1f70d	Oct 19	Bitmap large-run placement, decay on refill, empty-page trim
911d2ed	Oct 21	sigstorm, the signal-storm tool
c1a9791	Oct 21	Review round 2 fixes
fd735dd	Oct 21	Review round 3 fixes
7689931	Oct 21	Storm campaign 1: data and report
ae95875	Oct 22	Handler mappings for requests above 16 KiB (C1-1)
d2add24	Oct 23	Review round 4 fixes: the final allocator code
0b010d7	Nov 4	sohamalloc storm rerun on d2add24: data and report

C Where things are in the repository

Path	What
docs/IMPLEMENTATION_NOTES.md	The design as built: layouts, protocols, lock order, fork, deviations, limitations, the signal-safety guarantee (§10)
docs/STATUS.md	Current state and next steps
docs/research-log/01-timeline.md	Every phase, campaign and early defect, with times
docs/research-log/03-defects-and-faults.md	Review rounds 2–4 and C1-1: root causes, fixes, tests
docs/research-log/05-novelty-analysis.json	17 contributions against prior work, 50 references
docs/research-log/data/	Raw storm data, before/after records, test-matrix records
tools/sigstorm/	The signal-storm tool and campaign runner
bench/	Benchmark harness
tools/verify-matrix.sh	The full verification matrix in one command
docs/presentation/	This report, the slide deck and their sources